EQOWA FINTECH ENGINEERING DEPARTMENT

INSTITUTIONAL RWA PLATFORM FULL-STACK SYSTEM ARCHITECTURE

Comprehensive Engineering Specification & Execution Blueprint

Lead Architect: Ombre Knight

Target Chain: Polygon (Amoy & Mainnet EVM)

Core Standard: ERC-3643 (T-REX) & ONCHAINID

Classification: Confidential / Restricted Execution Spec

1. System Overview & Architecture Philosophy

EQOWA Fintech is engineering a production-grade, institutional-ready Real World Asset (RWA) tokenization and management engine. Traditional tokenization platforms fail because compliance is bolted on via superficial frontend toggles or centralized databases that easily drift from on-chain truth. Eqowa eliminates this risk by hard-coding identity registries and compliance modules directly into the asset smart contracts.

Core Architectural Principles

- **Smart-Contract Enforced Compliance:** No token transfer, mint, or burn can execute without cryptographic validation against the ONCHAINID identity registry and dynamic compliance modules.
- **Asynchronous State Reconciliation:** Decoupling high-frequency blockchain event logs from the PostgreSQL persistence layer using idempotent Node.js indexing pipelines.
- **Modular Institutional UX:** A lightning-fast Next.js App Router frontend tailored for institutional administrators, compliance officers, and accredited investors.

1.2 Technology Stack Breakdown

Layer	Technology	Purpose
Smart Contracts	Solidity 0.8.24, OpenZeppelin, ERC-3643 (T-REX)	Token issuance, identity registries, transfer rules.
Blockchain Network	Polygon PoS (Amoy Testnet / Mainnet)	High throughput, low-fee EVM finality.
Testing Framework	Foundry (forge / cast)	Fuzz testing, invariant simulation, reentrancy defense.
Backend Services	Node.js, TypeScript, Express / NestJS	API gateway, KYC webhook processing, state indexers.
Database & State	PostgreSQL, Redis	Relational transactional storage, event queues, caching.
Frontend Interface	Next.js (App Router), TypeScript, Tailwind CSS, Viem/Wagmi	Institutional dashboards, portfolio views, admin portals.

2. Smart Contract & ERC-3643 Core Specification

The token smart contract relies on the ERC-3643 (T-REX) standard, separating token ownership from identity validation and compliance rules. This ensures that regulatory updates (such as jurisdiction blocks or revoked KYC credentials) take immediate effect without redeploying the core token contract.

2.1 Core Contracts & Relationships

- **Token Contract (ERC-3643):** Represents the underlying real-world asset. Every transfer calls `canTransfer()` on the compliance module before updating balances.
- **Identity Registry (`IdentityRegistry.sol`):** Maintains the mapping between investor wallet addresses and their corresponding ONCHAINID smart contract storage (containing cryptographic claims).
- **Compliance Module (`Module.sol`):** Implements country-code restrictions, maximum wallet holding caps, and lock-up periods.
- **Trusted Issuers Registry:** Defines which KYC/AML verification authorities are authorized to sign claims on investor ONCHAINIDs.

2.2 Enforcing Transfer Restrictions

When an investor initiates a transfer, the following execution flow occurs on-chain:

1. Sender calls `transfer(to, amount)` on the T-REX token contract.
2. Token contract queries `IdentityRegistry.isVerified(sender)` and `isVerified(recipient)`.
3. If verified, the token contract queries the active `Compliance.canTransfer(sender, recipient, amount)` module.
4. The compliance module verifies country codes (e.g., restricting prohibited jurisdictions) and wallet balance caps.
5. If all conditions pass, execution proceeds; otherwise, the transaction reverts with a descriptive error selector.

2.3 Foundry Testing Invariants & Security Boundaries

Contracts must pass rigorous invariant and fuzz testing suites written in Foundry prior to deployment:

- **Reentrancy Defense:** Malicious receiver contracts attempting callback reentrancy during mint/burn/transfer cycles must be blocked by standard ReentrancyGuards and checks-effects-interactions patterns.

- **Forced Transfer Resistance:** Unauthorized admin paths or malicious actors attempting forced transfers without proper recovery roles must trigger immediate transaction reverts.
- **Global Emergency Pause:** Activating the emergency pause state must instantly freeze all standard transfers, mints, and burns while preserving authorized administrative recovery actions.

3. Node.js Backend & PostgreSQL Database Architecture

The backend acts as the bridge between off-chain institutional KYC providers, administrative workflows, and the Polygon blockchain state. Ensuring zero data drift between on-chain events and database records is critical.

3.1 PostgreSQL Database Schema Design

- **`investors` Table:** Stores internal profile IDs, hashed KYC data references, wallet addresses, country codes, and ONCHAINID contract deployment status.
- **`assets` Table:** Tracks token contract addresses, total asset valuations, metadata URIs, and active compliance module flags.
- **`transactions` Table:** Logs all issuance, transfer, redemption, and freeze events with foreign keys linking senders, recipients, block numbers, and transaction hashes.
- **`sync_state` Table:** Persists the last processed block number to enable seamless crash recovery and incremental event indexing.

3.2 Idempotent Event Ingestion Pipeline

To eliminate data drift and handle RPC drops or webhook retries gracefully, the backend event indexer operates under strict idempotency rules:

- **Block Confirmation Threshold:** The indexer waits for a minimum confirmation depth ($C \ge 12$ blocks on Polygon) before committing transactional state changes to PostgreSQL.
- **Idempotent Handlers:** Incoming event signatures (e.g., `Transfer`, `VerificationUpdated`) are checked against transaction hashes stored in the database. Duplicate events are safely ignored.
- **Reconciliation Cron:** An automated background worker runs every 15 minutes, comparing on-chain balances and registry states against database totals, automatically flagging discrepancies.

3.3 KYC & Compliance Webhook Integration

When an institutional investor completes onboarding via third-party providers (e.g., Sumsub, Fractal, or enterprise custody solutions), an HMAC-secured webhook hits the Node.js API. The backend service automatically invokes `registerIdentity()` or updates country claims on the Polygon ONCHAINID registry via a secure relayer wallet.

4. Next.js App Router & Institutional Frontend

The user-facing application is built using Next.js (App Router), TypeScript, and Tailwind CSS. It is split into two distinct user portals: the **Investor Portal** and the **Compliance Admin Dashboard**.

4.1 Architecture & Wallet Integration

- **State Management:** Powered by `wagmi`, `viem`, and TanStack React Query for lightning-fast blockchain state reads and reactive UI updates.
- **Authentication:** Hybrid authentication combining SIWE (Sign-In with Ethereum) for Web3 wallet verification and JWT session cookies for backend API authorization.
- **Network Guarding:** Automatic detection and prompt injection forcing users onto Polygon PoS (Chain ID 137 / Amoy 80002), preventing accidental mainnet or incorrect network transactions.

4.2 Portal Interface Specifications

Portal	Key Views & Features	Security / UX Focus
Investor Dashboard	Portfolio valuation, tokenized asset breakdown, dividend/yield payout history, KYC status tracker.	Clean, simplified institutional overview hiding complex blockchain gas mechanics.
Compliance Admin Portal	Investor whitelist management, manual KYC approval overrides, global emergency pause toggle, transfer audit logs.	Multi-signature requirement warnings, role-based access control (RBAC) visibility.

4.3 Error Handling & Transaction States

Web3 interfaces often fail due to confusing transaction rejections. The frontend implements robust transaction lifecycle handling:

- **Pre-flight Compliance Check:** Before submitting a transfer or mint transaction, the frontend queries `canTransfer()` statically via RPC to instantly display compliance errors (e.g., "Recipient not whitelisted") before the user pays gas.
- **Real-time Toast Notifications:** Clear visual feedback for transaction broadcast, block inclusion, confirmation depth, and explicit error translation if a smart contract reverts.

5. Testing Rigor & DevOps Deployment Protocol

Institutional RWA platforms cannot afford smart contract vulnerabilities or data corruption. The engineering lifecycle mandates a rigorous testing and staging protocol prior to mainnet promotion.

5.1 Multi-Stage Testing Framework

- **Unit & Integration Testing (Foundry):** 100% test coverage across all T-REX compliance modules, token transfer restrictions, and identity registry modifications.
- **Fuzz & Invariant Testing:** Simulating thousands of randomized transaction sequences to prove that total supply invariants hold, user balances never exceed compliance caps, and unauthorized addresses cannot bypass identity checks.
- **Staging Deployment (Polygon Amoy):** Full end-to-end integration testing connecting the Next.js frontend, Node.js indexer, and smart contracts on the Polygon Amoy testnet under simulated high-load conditions.

5.2 Production Deployment Checklist

1. Multi-sig setup (Gnosis Safe) configured as the ultimate owner of all T-REX factory, registry, and compliance contracts.
2. Environment hardening for Node.js backend services (secure RPC endpoints, encrypted database connection strings, rate-limited API routes).
3. Frontend domain configuration with strict Content Security Policies (CSP) and SSL enforcement.
4. Final architecture handover documentation delivered to the lead architect.

6. Milestone Roadmap & Financial Allocation

To ensure high-velocity execution and strict quality control under the fixed budget framework, the project is divided into four sequential milestones. The developer executes against this precise pre-engineered blueprint, eliminating discovery delays.

6.1 Detailed Milestone Breakdown

Milestone	Scope & Deliverables	Timeline	Budget
Milestone 1 Smart Contract Core	Deployment of ERC-3643 token, ONCHAINID identity registries, compliance modules, and Foundry invariant test suite.	Week 1	**$1,000**
Milestone 2 Backend & Indexer	Node.js / TypeScript microservices, PostgreSQL schema setup, idempotent event ingestion pipelines, and KYC webhook handlers.	Week 2	**$800**
Milestone 3 Frontend Portals	Next.js App Router interfaces, Wagmi/Viem wallet integration, Investor Portfolio view, and Compliance Admin dashboard.	Week 3	**$800**
Milestone 4 Staging & Handover	Polygon Amoy end-to-end integration testing, bug fixes, security validation, and final code/documentation handover.	Week 4	**$400**
Total Engineering Execution Budget			**$3,000**

6.2 Execution Rules for Tomorrow's Interviews

To the Lead Architect (Ombre Knight): You hold the complete technical blueprint above. During your calls with Charl and Rodrigo tomorrow, do not negotiate on scope. Present this 6-page architectural specification as the immutable ground truth. Emphasize that because discovery is already complete, they are purely executing code against defined specs—justifying the **$3,000 total milestone cap** ($1,000 to start on Milestone 1).